

NAME: J SATYA SRIKAR

ROLL NO:A24126552259

BRANCH AND SECTION:CSM-B

COURSE NAME: FULL STACK WEB DEVELOPMENT

COURSE CODE:23CM4121

Experiment:03

Date:

Title: Write a program using JavaScript (arrays, functions)

Objective:

o study JavaScript arrays and functions by performing array operations, using built-in methods, creating different types of functions, and handling asynchronous data.

System Requirements:

- Hardware: Processor (e.g., Core i5), RAM (e.g., 8GB), Storage.
- Software: Operating System (e.g., Windows 11 / Linux), Code Editor (e.g., VS Code), Browser (e.g., Chrome), Runtime/Database (e.g., Node.js, MongoDB). [1, 2]

Source Code:

Arrays.js

```
//Nobe tire - Creation of an array
const arr = [1,2,3,4,5,6]

//Construction creation
const arr2 = new Array(1,2,3,4,5,6)

console.log(arr)
console.log(arr2)

arr.push(58)
```

```
console.log("pushed 58:"+[""+arr+""])
arr.pop()
console.log("poped:"+[""+arr+""])
arr.unshift(2)
console.log("unshifted 1:"+[""+arr+""])
arr.shift()
console.log("shifted 1:"+[""+arr+""])
```

```
//The Holy Trinity (Mid Tier)
```

```
//map function in array
```

```
const newarr = arr.map(
  (num)=>{return num * 10}
)
```

```
console.log("mapped arr"+newarr)
```

```
const booleanArr = arr.filter(
  (num)=>{return num%2 === 0}
)
```

```
console.log("booleanArr:"+booleanArr)
```

```
const total = arr.reduce((val1_starts_With_zero,val2_starts_with_first_element)=>{return})
```

```
//Finding Stuff
```

```
function greater(num){
  return num>1;
}
```

```
console.log("Find Elements greater than 1",arr.find(greater))
```

```
console.log("Find index for number greater than 1",arr.findIndex(greater))
```

```
console.log("Does 7 is there in arr",arr.includes(7))
```

```
console.log("index of 6 "+arr.indexOf(6))
```

```
//slicing & Dicing
```

```
let sliced_arr = arr.slice(1,3)
```

```
console.log("sliced array",sliced_arr)
let spliced_arr = arr.splice(3,3)
console.log("spliced arr ",spliced_arr)
let unsorted_arr = [4,3,5,1]
console.log("Unsorted array:",unsorted_arr)
console.log("sorted array",unsorted_arr.sort())
```

```
//God Mode
```

```
// Destructuring
```

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50
};
console.log("Before Destructuring:",person)
let {firstName, lastName} = person;
console.log("After Destructuring - firstName:",firstName)
console.log("After Destructuring - lastName:",lastName)
console.log("Both Names:",{firstName,lastName})
```

```
//Spread Operator
```

```
const first = [8,6,5,4,6,6]
const second = [1,2,4,5,6,7]
console.log("first and second",first,second)
const combined = [...first,...second]
console.log("Combined:",combined)
```

```
//flat
```

```
const Fa = [1,[3,4]]
console.log("Before flat:",Fa)
console.log("After Flat:",Fa.flat())
```

Functions.js

```
// Normal way of function
```

```
// You can call it up here!
```

```
console.log(greetOG("Bro"));
```

```
function greetOG(name) {
```

```
  return `Sup, ${name}`;
```

```
}
```

```
// Function Expression
```

```
const greetMillennial = function(name) {
```

```
  return `Hello, ${name}`;
```

```
};
```

```
console.log(greetMillennial("Bro"));
```

```
// Arrow Function (The Gen-Z Way)
```

```
const greetGenZ = (name) => {
```

```
  return `Yo, ${name}`;
```

```
};
```

```
// THE GIGACHAD ONE-LINER (Implicit Return)
```

```
// If it's just one line of code, you can drop the {} and the 'return' keyword.
```

```
const greetFast = name => `Yo, ${name} no cap`;
```

```
console.log(greetFast("Bro"));
```

```
// Built-in Function
```

```
// Math.max() is a built-in function.
```

```
// You just hand it numbers and it does the math behind the scenes.
```

```
const highestScore = Math.max(45, 120, 13);
```

```
console.log(highestScore); // Output: 120
```

```

// Put 'async' before the arrow to unlock god mode
const fetchServerData = async () => {
  console.log("1. Sending request to server...");

  // The 'await' keyword tells JS to pause THIS specific function
  // until the fetch finishes, while the rest of your app keeps running fine.
  const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
  const data = await response.json();

  console.log("2. Data secured!", data.name);
};

fetchServerData();
console.log("3. Doing other stuff while waiting...");

// Multi-paramterized functions
const trackStormSystem = (windSpeed, pressure, isLandfall) => {
  if (windSpeed > 119 && isLandfall === true) {
    return `Red Alert: Severe cyclone making landfall. Pressure at ${pressure}hPa.`;
  }
  return "System is stable. Keep monitoring.";
};

// When you call it, you MUST pass the data in the exact same order
const status = trackStormSystem(140, 950, true);
console.log(status);

```

STEPS TO EXECUTE:

- 1) Open vs code
- 2) Click on Terminal
- 3) Click on New Terminal
- 4) Type node <your .js file>
- 5) Click enter

OUTPUT:

Arrays.js

```
PS D:\CSM259(3-1)FS\CSM259(3-1)FS\Topic_Wise_programs\week 3\Src> node arrays.js
[ 1, 2, 3, 4, 5, 6 ]
[ 1, 2, 3, 4, 5, 6 ]
pushed 58:[1,2,3,4,5,6,58]
popped:[1,2,3,4,5,6]
unshifted 1:[2,1,2,3,4,5,6]
shifted 1:[1,2,3,4,5,6]
mapped arr10,20,30,40,50,60
booleanArr:2,4,6
Find Elements greater than 1 2
Find index for number greater than 1 1
Does 7 is there in arr false
index of 6 5
sliced array [ 2, 3 ]
spliced arr [ 4, 5, 6 ]
Unsorted array: [ 4, 3, 5, 1 ]
sorted array [ 1, 3, 4, 5 ]
Before Destructuring: { firstName: 'John', lastName: 'Doe', age: 50 }
After Destructuring - firstName: John
After Destructuring - lastName: Doe
Both Names: { firstName: 'John', lastName: 'Doe' }
first and second [ 8, 6, 5, 4, 6, 6 ] [ 1, 2, 4, 5, 6, 7 ]
Combined: [
  8, 6, 5, 4, 6,
  6, 1, 2, 4, 5,
  6, 7
]
Before flat: [ 1, [ 3, 4 ] ]
After Flat: [ 1, 3, 4 ]
```

Functions.js

```
PS D:\CSM259(3-1)FS\CSM259(3-1)FS\Topic_Wise_programs\week 3\Src> node functions.js
Sup, Bro
Hello, Bro
Yo, Bro no cap
120
1. Sending request to server...
3. Doing other stuff while waiting...
Red Alert: Severe cyclone making landfall. Pressure at 950hPa.
2. Data secured! Leanne Graham
PS D:\CSM259(3-1)FS\CSM259(3-1)FS\Topic_Wise_programs\week 3\Src>
```

Result: Successfully executed Arrays.js and functions.js using javascript